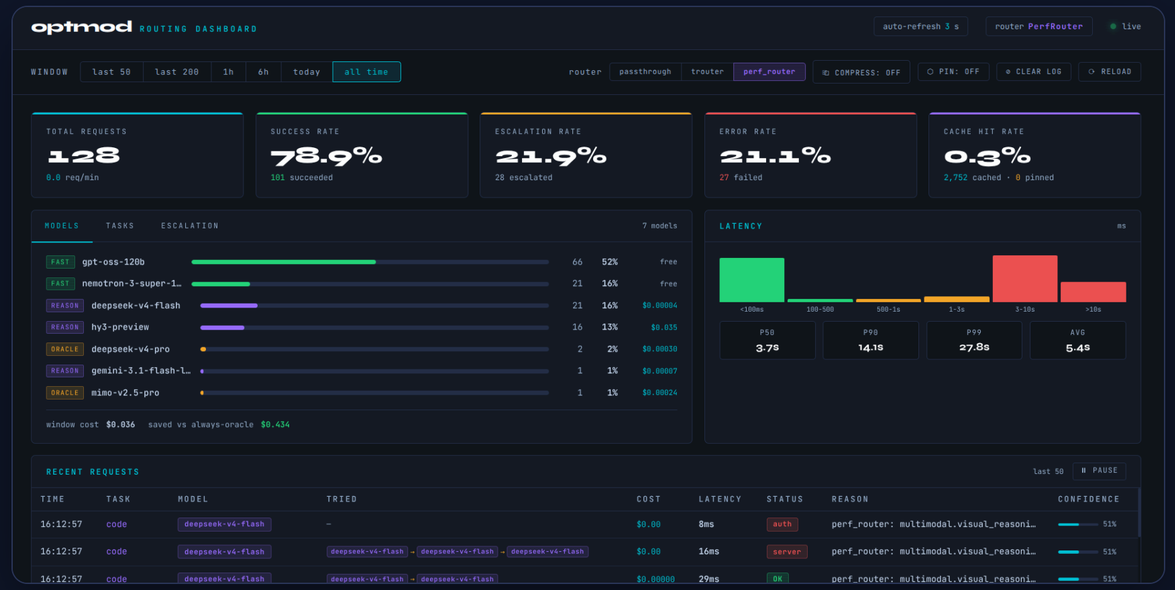
MODELROUTER · PROJECT OVERVIEW

optmod + PerfRouter

Route every LLM request to the model with the best predicted quality for its cost, with no LLM call to decide.



Goal 1

Best model per request

Predict each model's quality on the task and pick the best quality-for-cost, not one flagship for everything.

Goal 2

No routing overhead

Decide in 2–4 ms with sentence-BERT and XGBoost. No LLM call, no added cost, no extra latency.

Goal 3

Invisible to the agent

An OpenAI-compatible endpoint. Point any agent at it; routing, escalation and logging happen in one HTTP call.

github.com/jiuwencortex/optmod · github.com/jiuwencortex/perfrouter

ADVANTAGES

What the project delivers

No LLM in the loop

Task classification takes about 1 ms and a full routing decision 2–4 ms, all local.

Easy model onboarding

Models are scored from public Artificial Analysis and Arena data, so adding one needs no WildClawBench run.

Cost tuned at runtime

Price stays out of the trained model. Change the cost weight α or a quality floor without retraining.

Built-in resilience

Failed calls escalate fast → reasoning → oracle, and session pinning keeps multi-turn chats on cached-read pricing.

Interchangeable routers

Five routing algorithms swap at runtime with no restart, and the model pool is a config edit.

Observable decisions

Every request is logged to JSONL and shown on a live dashboard with cost, latency and routing reason.

TECHNICAL · OPTMOD

How a request flows through optmod

01	02	03	04	05
Extract	**Route**	**Mutate**	**Forward**	**Return**
Regex features from the request in under 1 ms	Five swappable routers. PerfRouter is the default	Adjust context: /think prefix, tool-output compression	Async call to DeepSeek or OpenRouter; escalate on error	Append to the JSONL log and return one clean response

Model pool and escalation

`fast ×3` → `reasoning ×3` → `oracle ×4`

10 models over DeepSeek direct and OpenRouter. Errors move a request up one tier, then 502.

Session pinning

`0–300 s`	**Hard pin.** Reuse the last model
`300–1800 s`	**Soft bonus.** Credit its cache savings
`over 1800 s`	**Fresh.** Normal routing

TECHNICAL · PERFROUTER

How PerfRouter picks a model

At request time, in 2–4 ms

01 **Classify.** sentence-BERT picks the top 3 of 33 task types.

02 **Predict.** XGBoost scores each model from 46 features.

03 **Decide.** Subtract the cost penalty; the best model passing the context, vision and cost-cap filters wins.

```
utility = quality - α × cost
```

α = 0.3, set at runtime

Built offline

0 Discover up to 50 models

1–2 Fetch AA scores and Arena ELO

3–5 Build weights and features

6 Train XGBoost, 10-fold CV

Labels. Benchmark affinity ×1, WildClawBench ground truth ×3.

Cost savings on WildClawBench

61 tasks · cheapest model within 25% of top quality · confirmed in real runs

80.1%

vs always V4 Flash, $0.175/1M

The 2× cost cap drops the premium models, so free or near-free models clear the bar on 74% of tasks.

32.2%

vs always V4 Pro, $0.544/1M

V4 Pro stays in the pool and sets the bar: 59% of tasks stay on it, the rest go to cheaper models.

WEDGES

Where it differs from other routers

Wedge	optmod + PerfRouter	Alternatives
New models	Scored from public benchmarks. Adding one is a YAML entry and a pipeline re-run.	RouteLLM and Not Diamond learn from prompt-level data; RouteLLM must be kept calibrated.
Model pool	Up to 60 models across providers, including free ones.	Bedrock routes between exactly two models from one family, English prompts only.
Quality in the gateway	Predicts quality per task type, with escalation and fallback built in.	LiteLLM, Portkey and OpenRouter route on fallback, load and price rules, not quality.
Token savings	Mutators shrink the request itself: RTK-style tool-output compression, plus cache-aware pinning.	Routers choose a model and forward the request unchanged.
Where it runs	Self-hosted. A 2–4 ms local decision, no markup, and prompts stay on your side.	Not Diamond and OpenRouter are managed APIs that receive every prompt.

ROADMAP

What comes next

Now

Consolidate

One config. optmod reads PerfRouter's model data directly.

Laya. Joins the decision chain alongside PerfRouter.

GAIA. Broader task types and difficulty levels.

Next

Learn from traffic

Feedback loop. Retrain routers from the routing log.

Durable pins. A file-backed session-pin store.

Baseline. Measure proxy overhead and export the model.

Later

Rust data plane

In Rust. Proxy, mutators, pinning and routing inference.

In Python. Training pipeline and model discovery.

Why. Throughput, footprint and single-binary deployment.

Interchangeable by design Routing algorithms and models plug in behind one interface. Each step swaps a part; the agent's endpoint never changes.